

HACKATHON TASK LIST

VoiceGuardAI - Deepfake Detection Resources

✓ TASK 1: Lightweight Deepfake Detection

Resources & Free Platforms

► A. Open-Source GitHub Repositories (Models & Code)

1. Awesome Deepfake Detection (Master Collection)

Curated list of models, datasets, papers, and tools for deepfake detection.

🔗 <https://github.com/Daisy-Zhang/Awesome-Deepfakes-Detection>

2. PyTorch Deepfake Detection (Xception / MesoNet)

Classic deepfake video detection using FaceForensics++.

🔗 <https://github.com/HongguLiu/Deepfake-Detection>

3. EfficientNet-based Deepfake Detector + UI

Includes model + Gradio demo (great for inference + demo).

🔗 <https://github.com/TRahulsingh/DeepfakeDetector>

4. CNN-LSTM Deepfake Video Detector

ResNext + LSTM architecture for video-level classification.

🔗 https://github.com/Balaji-Kartheek/DeepFake_Detection

5. Audio Deepfake Detection (Speech-based)

Detects fake vs real voice using MFCCs and deep learning.

🔗 <https://github.com/sksmta/audio-deepfake-detection>

► B. Free & Public Datasets

1. FaceForensics++ (Most Used Benchmark)

High-quality real & fake face videos (multiple manipulation methods).

🔗 <https://github.com/ondyari/FaceForensics>

2. DeepFake Detection Challenge Dataset (DFDC)

Large-scale real vs fake video dataset by Facebook & partners.

🔗 <https://www.kaggle.com/c/deepfake-detection-challenge/data>

3. WildDeepfake Dataset

Real-world deepfakes scraped from the internet (harder, realistic).

🔗 <https://github.com/OpenTAI/wild-deepfake>

4. ASVspoof (Audio Deepfake Dataset)

Benchmark dataset for synthetic and spoofed speech detection.

<https://datashare.ed.ac.uk/handle/10283/3336>

5. Fake-or-Real (FoR) Audio Dataset

Synthetic vs real speech dataset.

<https://github.com/mohammedsharaf/for-dataset>

► C. Free Compute / GPU / TPU Platforms

1. Google Colab (FREE GPU/TPU)

Free NVIDIA T4 GPU - Ideal for training & fine-tuning.

<https://colab.research.google.com/>

2. Kaggle Notebooks (FREE GPU)

Free GPU hours - Easy dataset hosting.

<https://www.kaggle.com/code>

3. Hugging Face Spaces (FREE Model Demos)

Deploy Gradio / Streamlit apps - Free ZeroGPU tier.

<https://huggingface.co/spaces>

4. Paperspace Gradient (Limited Free Tier)

Jupyter notebooks with GPU access.

<https://www.paperspace.com/gradient>

► D. Free Demo & UI Hosting

1. Streamlit Community Cloud

Free hosting for ML dashboards & demos.

<https://streamlit.io/cloud>

2. Gradio

Quick UI for ML inference (works with Hugging Face Spaces).

<https://gradio.app/>

3. Vercel / Netlify

For lightweight dashboards & landing pages.

<https://vercel.com/>

► E. Helpful Libraries (Free & Open Source)

1. PyTorch

Deep learning framework.

<https://pytorch.org/>

2. TensorFlow

Machine learning platform.

<https://www.tensorflow.org/>

3. OpenCV

Computer vision library.

<https://opencv.org/>

4. Librosa

Audio processing library.

<https://librosa.org/>

5. MediaPipe

Face landmarks detection.

<https://mediapipe.dev/>

✓ TASK 2: Deepfake Audio Detection Model

Detect real vs synthetic/cloned speech using MFCCs, spectrograms, or embeddings

► A. Open-Source GitHub Repositories (Audio Deepfake Detection)

1. Audio Deepfake Detection (MFCC + CNN)

Simple and clear implementation for fake vs real voice classification.

<https://github.com/sksmta/audio-deepfake-detection>

2. ASVspoof Baseline Systems (Official)

Reference implementations used in global spoof-detection challenges.

<https://github.com/asvspoof-challenge/2021>

3. SpeechBrain - Audio Spoof Detection Recipes

Production-grade speech ML framework with spoofing recipes.

<https://github.com/speechbrain/speechbrain>

4. Fake Speech Detection using Wav2Vec2

Fine-tuning pretrained speech models for fake audio detection.

<https://github.com/microsoft/DeepFakeDetection>

5. Whisper-Feature-Based Audio Deepfake Detection

Uses OpenAI Whisper embeddings for improved detection.

<https://github.com/piotrkawa/deepfake-whisper-features>

► B. Free & Public Audio Deepfake Datasets

1. ASVspoof 2019 / 2021 (MOST IMPORTANT)

Global benchmark dataset for fake speech & voice cloning.

<https://datashare.ed.ac.uk/handle/10283/3336>

2. Fake-or-Real (FoR) Audio Dataset

Synthetic vs real speech samples (easy to start with).

<https://github.com/mohammedsharaf/for-dataset>

3. WaveFake Dataset

GAN-generated speech + real speech pairs.

<https://github.com/nii-yamagishilab/wavefake>

4. LibriSpeech (Real Speech Dataset)

Used as clean 'real voice' baseline.

<https://www.openslr.org/12>

5. VCTK Voice Dataset

Multi-speaker real speech corpus.

 <https://datashare.ed.ac.uk/handle/10283/3443>

► C. Feature Extraction & Audio Libraries

1. Librosa

MFCC, Spectrograms extraction.

 <https://librosa.org/>

2. PyDub

Audio handling and manipulation.

 <https://github.com/jiaaro/pydub>

3. Torchaudio

PyTorch audio processing.

 <https://pytorch.org/audio/>

4. OpenSMILE

Audio feature extraction toolkit.

 <https://audeering.github.io/opensmile/>

► D. Free Compute / GPU Platforms

1. Google Colab (FREE GPU)

Perfect for training audio models.

 <https://colab.research.google.com/>

2. Kaggle Notebooks (FREE GPU)

Host datasets + training in one place.

 <https://www.kaggle.com/code>

3. Hugging Face Spaces (FREE Demo Hosting)

Deploy audio classifiers using Gradio.

 <https://huggingface.co/spaces>

4. Paperspace Gradient (Limited Free Tier)

Good alternative for longer runs.

 <https://www.paperspace.com/gradient>

► E. Demo & Visualization Tools

1. Gradio

Audio upload UI.

 <https://gradio.app/>

2. Streamlit

Audio dashboards.

 <https://streamlit.io/>

3. Hugging Face Transformers

Wav2Vec2, Whisper models.

 <https://huggingface.co/models>

► F. Suggested Starter Stack (Recommended)

1. Dataset: ASVspoof 2019
2. Features: MFCC / Spectrogram
3. Model: CNN / Wav2Vec2 fine-tuning
4. Training: Google Colab
5. Demo: Gradio + Hugging Face Spaces
6. Metrics: Accuracy, Precision, Recall, F1, EER



Important Notes for Participants:

- Training small models is sufficient
 - Using pretrained speech models is allowed
 - Focus on clean preprocessing + evaluation
 - Heavy datasets are not required
-

✓ TASK 3: Lip-Sync Mismatch Detection

Detect whether the lip movements in a video match the spoken audio

► A. Open-Source GitHub Repositories (Core Implementations)

1. Wav2Lip (MOST IMPORTANT)

State-of-the-art lip-sync model used as a baseline for sync comparison.

 <https://github.com/Rudrabha/Wav2Lip>

2. SyncNet (Audio-Visual Synchronization)

Classic deep learning model for measuring audio-video sync.

 https://github.com/joonson/syncnet_python

3. Lip-Sync Error Detection (Research Implementation)

Detects audio-visual mismatch using deep embeddings.

 <https://github.com/ahaliassos/LipSync>

4. Face Alignment & Landmark Detection

Helps track mouth movements frame-by-frame.

 <https://github.com/1adrianb/face-alignment>

5. OpenFace (Facial Behavior Analysis Toolkit)

Extracts facial landmarks and mouth region movement features.

 <https://github.com/TadasBaltrusaitis/OpenFace>

► B. Free & Public Datasets (Audio-Visual)

1. LRS2 / LRS3 (Lip Reading Sentences)

Large-scale datasets for audio-visual speech analysis.

 https://www.robots.ox.ac.uk/~vgg/data/lip_reading/

2. VoxCeleb (Talking Face Videos)

Thousands of real talking-head videos.

 <https://www.robots.ox.ac.uk/~vgg/data/voxceleb/>

3. FakeAVCeleb Dataset

Deepfake videos with manipulated audio-visual sync.

 <https://github.com/DASH-Lab/FakeAVCeleb>

4. YouTube Talking-Head Clips

Public videos for creating artificial mismatches.

 <https://www.youtube.com/>

► C. Audio & Video Processing Libraries

1. OpenCV

Video frame extraction.

 <https://opencv.org/>

2. FFmpeg

Audio-video separation.

 <https://ffmpeg.org/>

3. Librosa

Audio features.

 <https://librosa.org/>

4. MediaPipe Face Mesh

Mouth landmarks detection.

 <https://developers.google.com/mediapipe>

► D. Free Compute / GPU Platforms

1. Google Colab (FREE GPU)

Perfect for running Wav2Lip / SyncNet models.

 <https://colab.research.google.com/>

2. Kaggle Notebooks

Good for preprocessing and experimentation.

 <https://www.kaggle.com/code>

3. Paperspace Gradient

Alternative free notebook environment.

 <https://www.paperspace.com/gradient>

► E. Demo & Visualization Tools

1. Gradio

Video + Audio upload UI.

 <https://gradio.app/>

2. Streamlit

Score visualization.

 <https://streamlit.io/>

3. Matplotlib / Plotly

Sync score graphs.

 <https://matplotlib.org/>

► F. Suggested Approaches (Students Can Choose One)

1. Approach 1: SyncNet-based

Extract embeddings from audio & video, compute cosine distance, threshold for real vs mismatch.

2. Approach 2: Landmark-based

Track mouth landmarks, compare movement timing vs audio energy.

3. Approach 3: Artificial Mismatch Generation

Shift audio by ± 200 –500 ms, train a classifier to detect mismatch.

Important Notes for Participants:

- *You do not need to build a perfect lip-reading model*
 - *Simple mismatch scoring + explanation is enough*
 - *Creativity and reasoning matter more than accuracy*
-

✓ TASK 4: Adversarial Deepfake Challenge

Fool a deepfake detector using perturbations, then propose & implement a mitigation

► A. Open-Source GitHub Repositories (Adversarial ML Core)

1. Adversarial Robustness Toolbox (ART)

Industry-grade library for adversarial attacks & defenses.

 <https://github.com/Trusted-AI/adversarial-robustness-toolbox>

2. CleverHans

Classic adversarial ML library used in research & competitions.

 <https://github.com/cleverhans-lab/cleverhans>

3. AdverTorch (PyTorch-Focused)

Simple PyTorch-native adversarial attack & defense library.

 <https://github.com/BorealisAI/adversarial-attacks-pytorch>

4. Foolbox

Lightweight adversarial attack framework supporting many models.

 <https://github.com/bethgelab/foolbox>

5. Audio Adversarial Examples (Speech-Focused)

Research implementations for attacking speech recognition systems.

 https://github.com/carlini/audio_adversarial_examples

► B. Baseline Deepfake Models to Attack

1. EfficientNet Deepfake Detector

 <https://github.com/TRahulsingh/DeepfakeDetector>

2. XceptionNet / CNN-based Detector

 <https://github.com/HongguLiu/Deepfake-Detection>

3. Audio Deepfake Detector

 <https://github.com/sksmta/audio-deepfake-detection>

► C. Datasets (Reusable from Task 1 / 2)

1. FaceForensics++

Video dataset.

 <https://github.com/ondyari/FaceForensics>

2. DFDC (Kaggle)

Video dataset.

 <https://www.kaggle.com/c/deepfake-detection-challenge/data>

3. ASVspoof 2019 / 2021

Audio dataset.

 <https://datashare.ed.ac.uk/handle/10283/3336>

4. WaveFake

Audio dataset.

 <https://github.com/nii-yamagishilab/wavefake>

► D. Common Adversarial Techniques

1. Attacks

Gaussian noise, compression artifacts, random cropping, audio pitch shifting, time stretching, FGSM/PGD perturbations.

2. Defenses

Input preprocessing, data augmentation, ensemble models, confidence thresholding, feature-space regularization.

► E. Audio & Video Processing Tools

1. OpenCV

Video perturbations.

 <https://opencv.org/>

2. FFmpeg

Compression & distortions.

 <https://ffmpeg.org/>

3. Librosa

Audio perturbations.

 <https://librosa.org/>

4. Torchaudio

Audio processing.

 <https://pytorch.org/audio/>

► F. Threat Modeling & Reference Docs

1. MITRE ATLAS (Adversarial ML Threat Matrix)

Industry-recognized framework for ML attacks.

 <https://atlas.mitre.org/>

2. NIST Adversarial ML Taxonomy

Official U.S. standards on ML attack surfaces.

 <https://www.nist.gov/itl/ai-risk-management-framework>

Important Notes for Participants:

- *You do not need complex attacks*
 - *Simple perturbations + reasoning is enough*
 - *Focus on why the model fails and how to harden it*
-

✓ TASK 5: VoiceGuardAI Inference API

Build an API that accepts media → runs detection (real or mock) → returns results

► A. Backend Frameworks (API Core)

1. FastAPI (RECOMMENDED)

Modern, fast Python framework with automatic Swagger docs.

 <https://fastapi.tiangolo.com/>

2. Flask

Lightweight Python web framework for simple APIs.

 <https://flask.palletsprojects.com/>

3. Express.js (Node.js)

Popular JS backend framework.

 <https://expressjs.com/>

► B. ML Model Serving / Inference Wrappers

1. BentoML

Turn ML models into production-ready APIs easily.

 <https://github.com/bentoml/BentoML>

2. TorchServe (PyTorch Models)

Official PyTorch model server for inference.

 <https://github.com/pytorch/serve>

3. Hugging Face Inference

Load and run pretrained models easily.

 <https://huggingface.co/docs/transformers/index>

► C. File Upload & Media Handling

1. Python Multipart Uploads (FastAPI)

 <https://fastapi.tiangolo.com/tutorial/request-files/>

2. FFmpeg (Audio/Video processing)

 <https://ffmpeg.org/>

3. OpenCV (Frame extraction)

 <https://opencv.org/>

4. Librosa (Audio features)

 <https://librosa.org/>

► D. Authentication & Security

1. FastAPI JWT example

 <https://github.com/IndominusByte/fastapi-jwt-auth>

2. Flask JWT

 <https://flask-jwt-extended.readthedocs.io/>

3. SlowAPI (FastAPI rate limiting)

 <https://github.com/laurentS/slowapi>

4. Express Rate Limit

 <https://github.com/nfriedly/express-rate-limit>

► E. API Documentation & Testing

1. Swagger / OpenAPI

 <https://swagger.io/>

2. Postman (API testing)

 <https://www.postman.com/>

3. HTTPie (CLI testing)

 <https://httpie.io/>

► F. Free Deployment / Hosting Options

1. Render (FREE Tier)

Deploy FastAPI / Flask apps easily.

 <https://render.com/>

2. Railway (FREE Credits)

Good for quick backend demos.

 <https://railway.app/>

3. Hugging Face Spaces (API + Demo)

Host inference APIs with Gradio/Streamlit.

 <https://huggingface.co/spaces>

4. Google Cloud Run (FREE Tier + Credits)

Serverless container hosting.

 <https://cloud.google.com/run>

Important Notes for Participants:

- Mock models are allowed (random or rule-based outputs are OK)
 - Focus on clean API design, error handling, and documentation
 - Performance optimizations are a bonus, not mandatory
-

✓ TASK 6: End-to-End Mini Detection System

Upload media → API processes → results shown on UI

► A. Frontend Frameworks (UI Layer)

1. React.js

Most common framework for building dashboards.

 <https://react.dev/>

2. Next.js

Production-ready React framework with routing and APIs.

 <https://nextjs.org/>

3. Streamlit (Fastest ML UI)

Ideal for quick end-to-end ML demos.

 <https://streamlit.io/>

4. Gradio

Quick UI for media upload + inference results.

 <https://gradio.app/>

► B. Backend / API (Reuse from Task 5)

1. FastAPI

 <https://fastapi.tiangolo.com/>

2. Flask

 <https://flask.palletsprojects.com/>

3. Express.js

 <https://expressjs.com/>

► C. Example End-to-End Starter Repositories

1. FastAPI + React Full Stack Template

 <https://github.com/tiangolo/full-stack-fastapi-postgresql>

2. ML Model + Gradio Demo

 <https://github.com/gradio-app/gradio>

3. Streamlit ML App Examples

 <https://github.com/streamlit/example-apps>

4. Next.js Dashboard Starter

 <https://github.com/vercel/nextjs-dashboard>

► D. Model Integration

1. Real Models (Optional)

Use any model from Task 1 / Task 2.

2. Mock Model Options

Random confidence generator, rule-based classifier, pre-saved inference outputs.

► E. UI Components & Dashboard Templates

1. Shadcn UI

 <https://ui.shadcn.com/>

2. Material UI

 <https://mui.com/>

3. Chakra UI

 <https://chakra-ui.com/>

4. Devias Kit (Free)

 <https://github.com/devias-io/material-kit-react>

► F. Free Hosting & Deployment

1. Vercel (Frontend)

 <https://vercel.com/>

2. Netlify (Frontend)

 <https://www.netlify.com/>

3. Render (Backend)

 <https://render.com/>

4. Railway (Backend)

 <https://railway.app/>

Important Notes for Participants:

- *Mock models are acceptable*
 - *End-to-end working flow matters more than accuracy*
 - *Clean UI + stable API = high score*
-

✓ TASK 7: Secure Auth + Rate-Limited Gateway

Implement authentication, protected routes, and rate-limiting for media upload APIs

► A. Authentication (JWT / Token-Based Auth)

1. FastAPI JWT Authentication (Recommended)

Simple and production-friendly JWT auth implementation.

 <https://github.com/IndominusByte/fastapi-jwt-auth>

2. FastAPI Users (Full Auth System)

Complete auth system with JWT, OAuth, roles, and permissions.

 <https://github.com/fastapi-users/fastapi-users>

3. Flask JWT Extended

JWT auth support for Flask APIs.

 <https://flask-jwt-extended.readthedocs.io/>

4. Auth0 (Free Tier)

Managed authentication (social login, JWT, RBAC).

 <https://auth0.com/>

► B. Role-Based Access Control (RBAC)

1. FastAPI RBAC Example

 <https://github.com/00-Python/FastAPI-Role-and-Permissions>

2. Flask Role Permissions

 <https://flask-security-too.readthedocs.io/>

► C. Rate Limiting & Abuse Protection

1. SlowAPI (FastAPI)

Rate limiting using Redis or in-memory store.

 <https://github.com/laurentS/slowapi>

2. FastAPI Limiter

Redis-based high-performance rate limiting.

 <https://github.com/long2ice/fastapi-limiter>

3. Express Rate Limit (Node.js)

Rate limiting middleware for Express.

 <https://github.com/nfriedly/express-rate-limit>

► D. Input Validation & Security Middleware

1. Pydantic (FastAPI validation)

 <https://docs.pydantic.dev/>

2. OWASP Secure Headers

 <https://owasp.org/www-project-secure-headers/>

3. Helmet.js (Express security)

 <https://helmetjs.github.io/>

► E. API Security Best Practices & References

1. OWASP API Security Top 10

Common API vulnerabilities and mitigations.

 <https://owasp.org/API-Security/>

2. Rate-Limiting Patterns (Cloudflare Guide)

 <https://developers.cloudflare.com/rate-limiting/>

3. JWT Best Practices

 <https://jwt.io/introduction>

► F. Logging, Monitoring & Alerts (Optional Bonus)

1. Loguru (Python logging)

 <https://github.com/Delgan/loguru>

2. Prometheus (Metrics)

 <https://prometheus.io/>

3. Grafana (Visualization)

 <https://grafana.com/>

Important Notes for Participants:

- *Security design matters more than UI*
 - *Mock users and roles are acceptable*
 - *Rate limiting + auth = must-have*
 - *Clean explanations score higher than complex setups*
-

✓ TASK 8: Detection Dashboard (Web App)

Build a web UI to upload media, view detection results, confidence scores, and logs/history

► A. Frontend Frameworks (Core UI Layer)

1. React.js

Most widely used framework for building dashboards.

 <https://react.dev/>

2. Next.js

Production-grade React framework with routing and SSR.

 <https://nextjs.org/>

3. Vite (Fast Setup for React)

Lightweight and fast frontend tooling.

 <https://vitejs.dev/>

4. Streamlit (Python Alternative)

Quick dashboards for ML-focused UIs.

 <https://streamlit.io/>

► B. UI Component Libraries (Free & Open Source)

1. Shadcn UI (Highly Recommended)

Modern, clean UI components built on Tailwind + Radix.

 <https://ui.shadcn.com/>

2. Material UI (MUI)

Complete design system with tables, charts, modals.

 <https://mui.com/>

3. Chakra UI

Accessible, component-driven UI framework.

 <https://chakra-ui.com/>

4. Ant Design

Enterprise-style dashboard components.

 <https://ant.design/>

► C. Free Dashboard Templates (React-Based)

1. Devias Kit (Material UI)

Professional admin dashboard template.

 <https://github.com/devias-io/material-kit-react>

2. CoreUI Free React Admin

Bootstrap-based dashboard template.

 <https://github.com/coreui/coreui-free-react-admin-template>

3. Mantis (Ant Design)

Modern SaaS-style dashboard layout.

 <https://github.com/codedthemes/mantis-free-react-admin-template>

4. Tabler Dashboard

Simple, clean dashboard UI.

 <https://github.com/tabler/tabler>

► D. Charts, Tables & Visualization

1. Recharts

React charts.

 <https://recharts.org/>

2. Chart.js

 <https://www.chartjs.org/>

3. ECharts

 <https://echarts.apache.org/>

4. TanStack Table

Data tables.

 <https://tanstack.com/table>

► E. Media Upload & Preview

1. React Dropzone (Drag & Drop)

 <https://react-dropzone.js.org/>

2. Video.js (Video preview)

 <https://videojs.com/>

3. WaveSurfer.js (Audio waveform)

 <https://wavesurfer-js.org/>

► F. API Integration

1. Axios (HTTP client)

<https://axios-http.com/>

2. React Query / TanStack Query

<https://tanstack.com/query>

Important Notes for Participants:

- Backend can be real or mocked
 - UI clarity > visual complexity
 - Clean UX + usable dashboard scores higher
 - Accessibility and responsiveness are bonuses
-

✓ TASK 9: VoiceGuardAI Landing Page

Create a modern product website explaining VoiceGuardAI, features, use-cases, and a demo/CTA

► A. Frontend Frameworks (Static / SPA)

1. HTML + CSS + JavaScript

Great for clean, lightweight landing pages.

<https://developer.mozilla.org/en-US/docs/Web>

2. React.js

Reusable components and animations.

<https://react.dev/>

3. Next.js

SEO-friendly, production-ready framework.

<https://nextjs.org/>

► B. UI Libraries & Styling (Free)

1. Tailwind CSS (Highly Recommended)

Utility-first CSS for modern landing pages.

<https://tailwindcss.com/>

2. Bootstrap

Quick layout and responsiveness.

<https://getbootstrap.com/>

3. Framer Motion (Animations)

Smooth hero animations and transitions.

<https://www.framer.com/motion/>

► C. Free Landing Page Templates (Open Source)

1. Tailwind Landing Page Templates

Collection of clean SaaS landing pages.

 <https://github.com/tailwindtoolbox/Landing-Page>

2. Startup Landing Page (Next.js + Tailwind)

Modern startup-style template.

 <https://github.com/creativetimoofficial/tailwind-starter-kit>

3. SaaS Landing Page (React)

Simple SaaS homepage with sections & CTA.

 <https://github.com/saadpasta/react-landing-page-template>

4. Astro Landing Pages (Very Fast)

Static-first landing pages with great performance.

 <https://astro.build/themes/>

► D. Design & Content Resources

1. Heroicons

Icons.

 <https://heroicons.com/>

2. Lucide Icons

Icons.

 <https://lucide.dev/>

3. unDraw (Free SVGs)

Illustrations.

 <https://undraw.co/>

4. Google Fonts

Typography.

 <https://fonts.google.com/>

► E. Free Hosting & Deployment

1. Vercel (Best for Next.js)

 <https://vercel.com/>

2. Netlify

 <https://www.netlify.com/>

3. GitHub Pages

 <https://pages.github.com/>

► F. Suggested Landing Page Sections

1. Hero section

What VoiceGuardAI does.

2. Problem statement

Deepfake risk.

3. Product features

4. How it works

5. Demo / Screenshot section

6. Use cases

Banks, Law, Media, Individuals.

7. Call-to-action

Try Demo / Contact.

8. Footer

Links, socials.

Important Notes for Participants:

- Focus on clarity and storytelling
 - Mock demos/screenshots are allowed
 - Performance & mobile responsiveness matter
 - Clean design > heavy animations
-

✓ TASK 10: Mobile App Prototype for Deepfake Alerts

Design and/or build a mobile app that warns users about suspicious or fake audio/video

► A. Mobile App Frameworks (Free & Open Source)

1. Flutter (Highly Recommended)

Single codebase for Android + iOS, great UI.

 <https://flutter.dev/>

2. React Native

JavaScript-based mobile framework.

 <https://reactnative.dev/>

3. Expo (React Native Simplified)

Faster development with managed workflow.

 <https://expo.dev/>

► B. UI Design Tools (No-Code / Low-Code)

1. Figma (FREE)

Collaborative UI/UX design tool.

 <https://www.figma.com/>

2. Canva (Mobile Mockups)

 <https://www.canva.com/>

3. Adobe XD (Free Plan)

 <https://www.adobe.com/products/xd.html>

► C. Free Mobile UI Templates

1. Flutter UI Templates

 <https://github.com/mitesh77/Best-Flutter-UI-Templates>

2. React Native UI Kitten

 <https://akveo.github.io/react-native-ui-kitten/>

3. NativeBase (React Native)

 <https://nativebase.io/>

► D. Caller ID / Alert Inspiration

1. Truecaller-style UI patterns

Research how Truecaller shows spam/scam warnings.

2. Notification-based alerts

Use push notifications for deepfake warnings.

► E. Backend / API Integration

1. FastAPI

For backend detection API.

 <https://fastapi.tiangolo.com/>

2. Firebase (FREE Tier)

Backend-as-a-service for mobile apps.

 <https://firebase.google.com/>

3. Supabase (FREE Tier)

Open-source Firebase alternative.

 <https://supabase.com/>

► F. Free Deployment & Testing

1. Expo Snack (Online Flutter/React Native IDE)

 <https://snack.expo.dev/>

2. Firebase App Distribution

 <https://firebase.google.com/products/app-distribution>

3. TestFlight (iOS)

 <https://developer.apple.com/testflight/>

Important Notes for Participants:

- *Design-only submissions (Figma/Canva) are allowed*
 - *Prototype functionality matters more than polish*
 - *Mock detection results are acceptable*
 - *Focus on user experience and alert design*
-

✓ TASK 11: CI/CD Pipeline for Deepfake Processing

Automate testing, build, and deployment for a model/API pipeline

► A. CI/CD Platforms (Free / Open Source)

1. GitHub Actions (RECOMMENDED)

Free CI/CD for public repos, easy YAML workflows.

 <https://docs.github.com/en/actions>

2. GitLab CI/CD

Built-in CI/CD with free runners for public projects.

 <https://docs.gitlab.com/ee/ci/>

3. Jenkins (Open Source)

Self-hosted automation server (classic CI/CD).

 <https://www.jenkins.io/>

► B. Example CI/CD Repositories (Ready References)

1. FastAPI + GitHub Actions Example

 <https://github.com/tiangolo/full-stack-fastapi-postgresql>

2. Python CI Template (Tests + Lint)

 <https://github.com/actions/starter-workflows/tree/main/ci>

3. ML CI/CD Example (Train → Test → Deploy)

 <https://github.com/iterative/example-get-started>

► C. CI/CD Pipeline Components

1. PyTest

Testing framework.

 <https://docs.pytest.org/>

2. Coverage.py

Code coverage.

 <https://coverage.readthedocs.io/>

3. Black

Code formatter.

 <https://black.readthedocs.io/>

4. Flake8

Linting.

 <https://flake8.pycqa.org/>

5. Docker Build Action

Container builds.

 <https://github.com/docker/build-push-action>

► D. Model & Artifact Versioning

1. DVC (Data Version Control)

 <https://dvc.org/>

2. MLflow (Experiment Tracking)

 <https://mlflow.org/>

► E. Deployment Targets (Free Tier)

1. Render

 <https://render.com/>

2. Railway

 <https://railway.app/>

3. Hugging Face Spaces (CI + Demo)

 <https://huggingface.co/spaces>

Important Notes for Participants:

- Pipeline can be simple but complete
 - Screenshots/logs of successful runs are enough
 - Focus on automation, not complexity
-

✓ TASK 12: Dockerized Inference Environment

Containerize a model + API so it runs anywhere reproducibly

► A. Containerization Tools

1. Docker (CORE REQUIREMENT)

 <https://www.docker.com/>

2. Docker Compose (Multi-service apps)

 <https://docs.docker.com/compose/>

3. Podman (Docker Alternative)

 <https://podman.io/>

► B. Example Docker Repositories

1. FastAPI + Docker Template

 <https://github.com/tiangolo/uvicorn-gunicorn-fastapi-docker>

2. ML Model Docker Example

 https://github.com/ahmedfgad/ML_API

3. BentoML Dockerized Model

 <https://github.com/bentoml/BentoML>

► C. Writing Dockerfiles (Guides)

1. Official Dockerfile Best Practices

 https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

2. Python Docker Guide

 <https://docs.docker.com/language/python/>

► D. Model Serving Inside Containers

1. FastAPI + Uvicorn

 <https://fastapi.tiangolo.com/deployment/docker/>

2. TorchServe (Docker)

 <https://pytorch.org/serve/docker.html>

► E. Free Image Hosting / Registry

1. Docker Hub (Public repos)

 <https://hub.docker.com/>

2. GitHub Container Registry

 <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>

► F. Free Deployment (Containers)

1. Google Cloud Run (Free Tier)

 <https://cloud.google.com/run>

2. Render (Docker support)

 <https://render.com/>

3. Railway (Docker)

 <https://railway.app/>

Important Notes for Participants:

- Model can be mock or real
 - Docker build + run must work
 - Clear README instructions are mandatory
 - Smaller images & faster startup are bonuses
-

✓ TASK 13: High-Fidelity Detection Dashboard Design

Design a professional web dashboard UI for deepfake detection

► A. Design Tools (Free)

1. Figma (RECOMMENDED)

Industry-standard UI/UX design tool.

 <https://www.figma.com/>

2. Penpot (Open-Source Alternative)

Completely open-source design platform.

 <https://penpot.app/>

► B. Dashboard UI Inspiration & Design Systems

1. SaaS Dashboard Inspiration

 <https://www.saasframe.io/>

2. Dribbble - Dashboard Designs

 <https://dribbble.com/tags/dashboard>

3. Material Design 3

 <https://m3.material.io/>

4. IBM Carbon Design System

 <https://carbondesignsystem.com/>

► C. UI Kits (Free)

1. Figma Community - Dashboard UI Kits

 <https://www.figma.com/community>

2. Tailwind UI (Community / Free Kits)

 <https://tailwindui.com/components>

3. Ant Design Figma Kit

 <https://ant.design/docs/spec/introduce>

► D. Icons & Visual Assets

1. Lucide Icons

 <https://lucide.dev/>

2. Heroicons

 <https://heroicons.com/>

3. unDraw Illustrations

 <https://undraw.co/>

► E. Suggested Screens to Design

1. Media upload screen

2. Detection result screen

Confidence, label.

3. History / logs view

4. Analytics dashboard

5. Settings / profile

Important Notes for Participants:

- Focus on clarity, hierarchy, and UX flow
- Dark-mode friendly dashboards are a plus

- Explain design decisions briefly
-

✓ TASK 14: Mobile App UI Mockup for Deepfake Detection

Design mobile app screens showing alerts, detection results, and history

► A. Mobile Design Tools

1. Figma (Mobile Prototyping)

 <https://www.figma.com/>

2. Adobe XD (Free Tier)

 <https://www.adobe.com/products/xd.html>

► B. Mobile UI Guidelines & References

1. Material Design for Android

 <https://m3.material.io/>

2. iOS Human Interface Guidelines

 <https://developer.apple.com/design/human-interface-guidelines/>

► C. Mobile UI Kits (Free)

1. Figma Community - Mobile UI Kits

 <https://www.figma.com/community>

2. Flutter Mobile UI Kits

 <https://github.com/mitesh77/Best-Flutter-UI-Templates>

► D. App Inspiration

1. Truecaller App (UI Flow Reference)

 <https://play.google.com/store/apps/details?id=com.truecaller>

2. Call-Screen UX Patterns

 <https://material.io/design/platform-guidance/android-calls.html>

► E. Suggested Screens

1. Home / dashboard
2. Call alert screen (fake warning)
3. Detection result
4. History / logs
5. Settings

 Important Notes for Participants:

- Mockups are enough (no code required)
 - UX logic > visual complexity
 - Explain why alerts appear and how users respond
-

✓ TASK 15: QA Test Plan for Deepfake Detection Pipeline

Create test cases, scenarios, and minimal automation

► A. QA & Testing Frameworks (Free)

1. PyTest (Backend / API Testing)

<https://docs.pytest.org/>

2. Postman (API Testing)

<https://www.postman.com/>

3. Selenium (Web UI Testing)

<https://www.selenium.dev/>

4. Cypress (Modern Frontend Testing)

<https://www.cypress.io/>

► B. Test Case & QA Documentation Templates

1. QA Test Case Template (Google Docs)

<https://www.smartsheet.com/free-test-case-templates>

2. Bug Report Template

<https://www.atlassian.com/software/jira/templates/bug-tracking>

► C. API & Load Testing

1. Locust (Performance Testing)

<https://locust.io/>

2. JMeter (Load Testing)

<https://jmeter.apache.org/>

► D. CI Integration for QA (Optional Bonus)

1. GitHub Actions + PyTest

<https://docs.github.com/en/actions>

► E. Suggested QA Coverage Areas

1. File upload edge cases
2. Incorrect formats
3. Large file handling

4. False positive / false negative scenarios

5. API failure & timeout handling

6. UI error states

 Important Notes for Participants:

- *Manual test plans are acceptable*
 - *Automation is a bonus, not mandatory*
 - *Clear thinking > number of test cases*
-